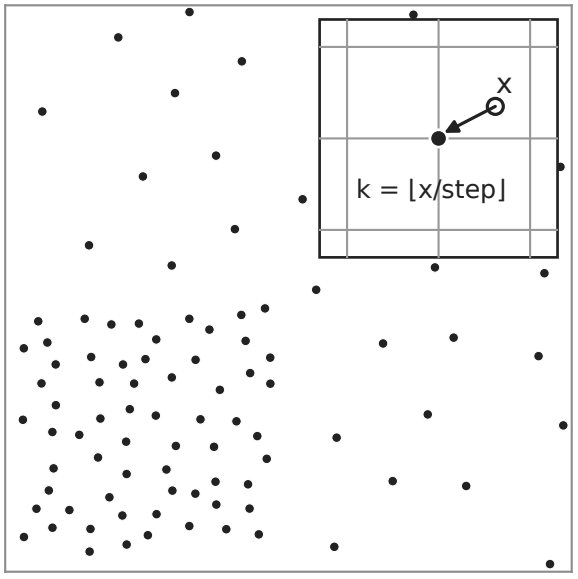
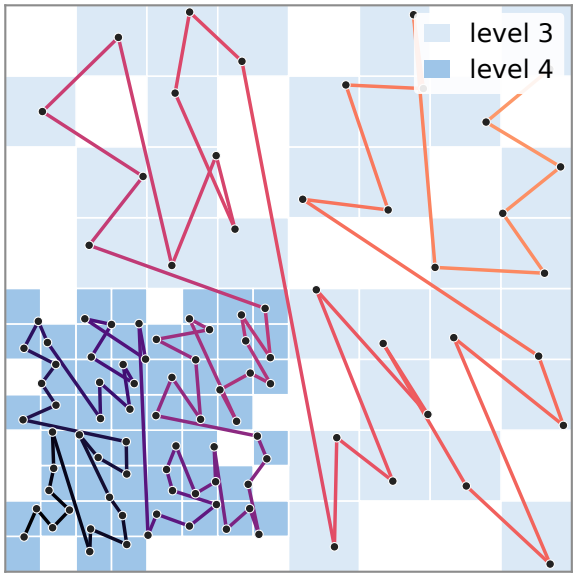
1 Snap to the grid GPU



Positions go down to an integer grid of 2^{30} steps per side (inset: one grid cell, much enlarged). Every later decision is an exact test on these integers; exact ties break by particle id.

`exact_predicates.hpp: snap`
`voronoi_mesh_2d.hpp: Mesh::snap`

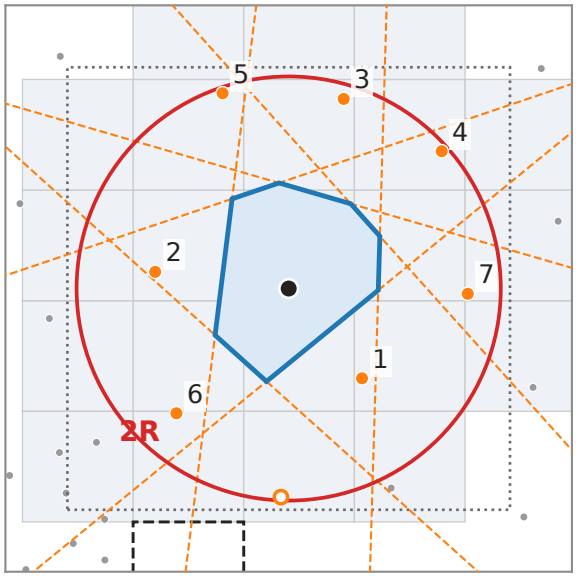
2 Morton cell list GPU



Sorted along a Morton curve (line, slot 0 to 95): each tree node is one run of slots. Each particle's level has about one particle per cell: 4 (16^2) in the patch, 3 (8^2) around.

`neighbor_search.hpp: configure, build`

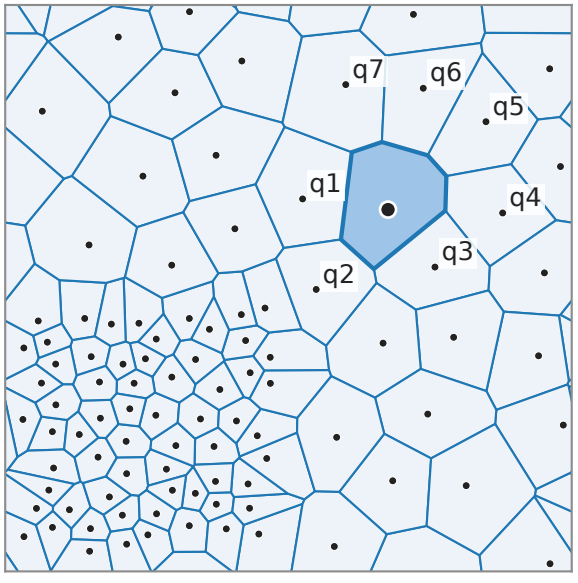
3 Clip one cell GPU



From a square of half-width L/4, cut by bisectors nearest first (numbers). The walk stops at the first grid cell 2R away (dashed), R the farthest corner: 7 cuts, 1 miss (hollow), 87 never read.

`polygon_clip.hpp: construct_pass`
`3D: shell_voronoi_warp.hpp`

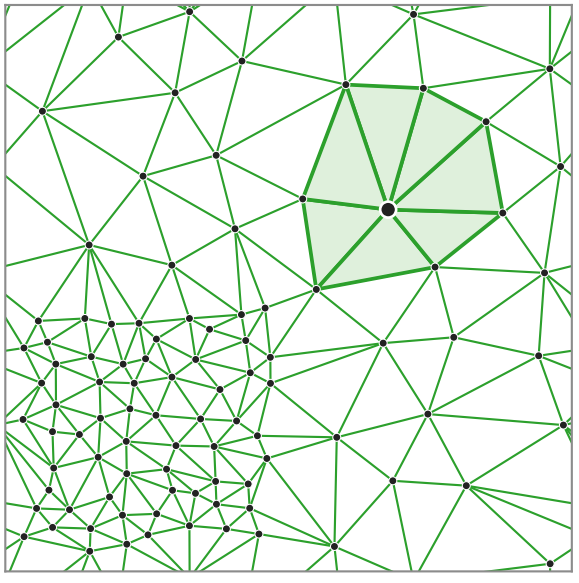
4 All cells: face rings GPU



Every cell is clipped on its own, in parallel, and keeps its ring: its neighbours in order (q1 … q7 here), in one pool. The rings are the whole topology; no corner is stored.

`polygon_clip.hpp: construct`
`3D: shell_voronoi.hpp, per face`

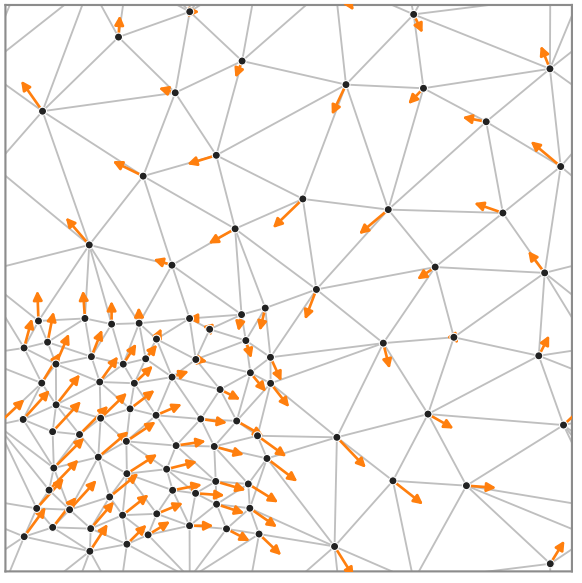
5 Exact start GPU



Consecutive ring entries give triangles (i, q_m, q_{m+1}), each once: 192 = 2n, with 3n = 288 faces. Checked: the exact screen and flips change 0 of them; the areas add up to the box once.

`tri_mesh.hpp: from_rings`
`3D: tet_start.hpp: build, check`

6 Points move caller



The solver calls update(x) with new positions, here up to 0.25 mean spacings (arrows ×3). The triangles are kept (grey); a step needs no cell list and no clipping.

`voronoi_mesh_2d.hpp: Mesh::update`
`(the caller: luna-cpp, every step)`